



IT Genius Engineering
IT GENIUS ENGINEERING CO., LTD.

COURSE ID
MOB-15

COURSE OUTLINE · HANDS-ON WORKSHOP

MOBILE / FULL-STACK

Basic to Advanced Laravel 13 and Flutter Framework

• 30 Hours • 5 Days • Level Basic → Advanced

HANDS-ON WORKSHOP
BASIC TO ADVANCED LARAVEL 13 AND FLUTTER FRAMEWORK
BUILD POWERFUL APIS, SHIP SMARTER APPS.

30 HOURS
5-DAY TRAINING
6 HOURS PER DAY

WHAT YOU WILL BUILD

- LOGIN & AUTH: Secure Access with Sanctum / JWT
- REAL-TIME DASHBOARD: Live Monitoring with WebSocket
- OFFLINE REPORTS: Cache & Sync Automatically
- INCIDENT REPORTING: Create, Track, and Resolve Issues
- DATA LOGGING: Real Time with BlobObserver

TECHNICAL STACK

- LARAVEL 13: RESTful API, Sanctum/CRT, API Resources, Repository Pattern, MySQL/PostgreSQL
- FLUTTER: Widget Tree, UI Components, Responsive Design, Mobile Best Practices
- RIVERPOD 3.0: Async/Await, AsyncNotifier, Mutable State, Automatic Berry
- CUBIT: Auth Flow, Business Logic, Audit Trail, BlobObserver, Enterprise Ready
- OFFLINE-FIRST: Persistent, Offline Caching, Background Sync, Works Anywhere

SAMIT KOYOM
Software Development & Mobile Application Design Specialist
15+ Years of Experience

2026 UPDATED

PREREQUISITES

- Basic Knowledge of PHP (OOP) and Introductory Dart/Flutter
- Basic SQL (MySQL or PostgreSQL)
- OS & Terminal basic skills
- No prior knowledge required of Riverpod, Cubit, Laravel Sanctum, or WebSockets

FORMAT: In-house Training or Online via Zoom

ONLY CODE: Hands-on coding with Starter Kit provided

SCAN TO ENROLL



Duration: 5 Days
30 hours total (6 hrs/day)



Format: In-house
or Online via Zoom



Course details
www.itgenius.co.th

Modern application development demands a cleanly layered architecture: a backend that serves data through APIs, and a mobile front end that responds instantly, keeps working under unstable network conditions, and stays maintainable over the long term. Choosing the right stack — and knowing how to use each tool for the right job — has become an essential skill for today's developers.

This course takes developers from basic to advanced, building a complete full-stack system end to end — from a **Laravel 13** API on MariaDB/PostgreSQL to a **Flutter** mobile app that uses **Riverpod 3.0 + Cubit** for state management. Learners write real code every day and assemble all the pieces into a complete monitoring-app workshop on the final day.

TECH STACK



Laravel 13



Flutter



Dart 3.x



PHP 8.3



PostgreSQL



MariaDB

TRAINING FEE · IN-HOUSE

Request a Quote

Flat in-house package · dates and group size by arrangement · quote available on request.



ADD
LINE

02-570-8449

088-807-9770

@itgenius

www.itgenius.co.th



01 Course Objectives Objectives

- Design and build RESTful APIs with Laravel 13, complete with authentication (Sanctum/JWT) and the Repository Pattern on MariaDB/PostgreSQL
- Understand the Flutter widget tree and build UI components for a real dashboard
- Apply Riverpod 3.0 for async state management — AsyncValue, AsyncNotifier, and the new Mutations API
- Use Cubit for enterprise-grade business logic such as auth sessions, status management, and audit logging
- Build an offline-first mobile app that caches data and syncs back when reconnected
- Walk away with an application prototype ready to extend into production

02 Who Is This Course For? Target Audience

- Software developers who want to build or maintain full-stack systems with Laravel and Flutter
- Backend developers with a PHP/Laravel foundation who want to connect their API to a Flutter app
- Mobile developers who want advanced state management with Riverpod 3.0 and Cubit in a real project
- Junior developers new to Flutter who want to learn state management in a real project from the start

03 Prerequisites Prerequisites

- Programming basics: PHP (basic OOP) and introductory Dart/Flutter, or experience building other apps
- Basic SQL (SELECT, INSERT, JOIN) on MariaDB or PostgreSQL
- Git & Terminal: able to clone, commit, push, and run command-line commands
- No prior knowledge required of Riverpod, Cubit, Laravel Sanctum, or WebSocket — taught from the ground up

04 Course Highlights Why This Course

- **Hybrid State Management (Riverpod 3.0 + Cubit):** Riverpod for the data layer and async state, Cubit for complex business logic such as auth flow and audit trails
- **Offline-First Architecture:** the app keeps working when the network drops, using riverpod_sqflite caching that syncs when back online
- **Laravel 13 + Repository Pattern:** clean, testable APIs that switch between MariaDB and PostgreSQL without touching business logic

05 Training Roadmap 5-Day Map

- **DAY 1** · Laravel 13 API Layer + Riverpod Async State
- **DAY 2** · Flutter Foundation — Widget System & Navigation
- **DAY 3** · Riverpod 3.0 Fundamentals — Providers & Consumer Patterns
- **DAY 4** · Advanced Riverpod + Cubit for Enterprise Logic
- **DAY 5** · End-to-End Workshop — Monitoring App (all 5 features)



DAY
1

Laravel 13 API Layer + Riverpod Async State

Training Topics · Section 1-2

TOTAL
6 Hours

1

Laravel 13 API Layer

- **Project Setup + API Architecture:** install Laravel 13 with the Repository Pattern — separate Controller, Service, and Repository for clean, testable code, with API versioning (/api/v1/)
- **Authentication with Sanctum + JWT:** design the Login/Logout flow, token-based auth for mobile clients, and middleware guards for protected routes
- **API Resources & Collections:** transform Eloquent models into clean JSON with Resource Classes, nested collections, and conditional fields — preventing over-fetching
- **MariaDB / PostgreSQL Repository Pattern:** a repository interface and implementation that switches drivers without changing business logic — migrations, seeders, and transactions

2

Riverpod 3.0 Async State

- **AsyncValue:** the state machine (data / loading / error) and how to consume API results with `when()` / `guard()` in a null-safe way
- **AsyncNotifier + Mutations API (3.0):** build an AsyncNotifier for CRUD using the new Mutations API with the lifecycle `idle` → `pending` → `success/error`
- **Automatic Retry:** add retry logic with Riverpod's `retry` parameter and a custom exception handler for unstable networks
- **Day 1 Lab:** build a Laravel API endpoint with Sanctum auth, plus a Flutter widget that fully renders the AsyncValue Loading / Error / Data states



DAY
2

Flutter Foundation — Widget System & Navigation

Training Topics · Section 3–4

TOTAL
6 Hours

3 Flutter Architecture Overview

- **Rendering Pipeline:** understand the Widget Tree, Element Tree, and RenderObject Tree — and why Flutter rebuilds faster than native in many cases
- **Stateless vs Stateful + setState:** build UI with StatefulWidget until the "problems" become clear — scattered state, prop-drilling, and unnecessary rebuilds
- **Widget Tree & BuildContext:** use BuildContext correctly, the differences between contexts at each level, and how to avoid common context bugs

4 Layout & Navigation

- **Layout Widgets:** Column, Row, Stack, Expanded, Flexible, ListView.builder, GridView.builder — a responsive dashboard layout for phones and tablets
- **Custom Widgets + Composition:** break UI into small reusable widgets such as StatCard, StatusBadge, AlertTile — practicing DRY
- **Navigation + GoRouter:** named routes, route guards (auth check), deep links, and bottom navigation
- Day 2 Lab: build a Flutter app skeleton — Login, Dashboard, Reports, and Settings screens with complete navigation and at least 3 custom widgets



DAY
3

Riverpod 3.0 Fundamentals — Providers & Consumer Patterns

Training Topics · Section 5–6

TOTAL
6 Hours

5 ProviderScope & Provider Types

- **Why State Management?** revisit the Day 2 setState problems and compare code before/after Riverpod — prop-drilling, unnecessary rebuilds, and testability
- **ProviderScope + Lifecycle:** set up ProviderScope at the root, auto-dispose, keep-alive, and how to override providers in a test environment
- **Basic Provider Types:** Provider (constants/singletons), StateProvider (filter/toggle), and FutureProvider (read-only API calls)

6 Consumer Patterns & Code Generation

- **ConsumerWidget & ConsumerStatefulWidget:** write widgets that subscribe to Riverpod providers correctly
- **ref.watch vs ref.read vs ref.listen:** the most important distinction in Riverpod — watch (rebuild), read (one-time), listen (side effect) + anti-patterns to avoid
- **@riverpod Annotation + Code Generation:** use riverpod_generator and riverpod_annotation for type-safe providers and less boilerplate
- Day 3 Lab: connect the Flutter app to the Day 1 Laravel API with FutureProvider and AsyncNotifier — show real data on the dashboard with a loading skeleton and error banner



DAY
4

Advanced Riverpod + Cubit for Enterprise Logic

Training Topics · Section 7–8

TOTAL
6 Hours

7

Riverpod Advanced

- **NotifierProvider:** for state with multiple fields and methods, such as a report state with filter, sort, pagination, and refresh
- **StreamProvider + WebSocket Real-time:** connect to a Laravel WebSocket (Pusher/Ably or Laravel Echo) and display real-time data
- **Provider Family:** create providers that take parameters such as `detailProvider(id)` for the list/detail pattern
- **Offline Persistence (riverpod_sqflite):** cache data into SQLite — define TTL, sync strategy, and conflict resolution when back online

8

Cubit — Enterprise Logic

- **Cubit vs BLoC:** compare the two and why Cubit is chosen — simpler and well-suited to enterprise logic at this scale
- **AuthCubit (Login / Logout / Session):** secure token storage (`flutter_secure_storage`), auto-refresh token, session timeout, app-wide logout
- **StatusCubit + BlocObserver:** track state with a `BlocObserver` that logs every transition as an audit trail
- Day 4 Lab: add a real-time dashboard with `StreamProvider + WebSocket`, an `AuthCubit` for login, and offline cache — test by turning off the network and observing the fallback



DAY
5

End-to-End Workshop — EDL-Generation Monitoring App

Training Topics · Section 9–10

TOTAL
6 Hours

9 Workshop — Build the 5 App Features

- **Feature 1 — Login & Auth (Cubit + Sanctum):** assemble AuthCubit with the Sanctum API — Login, Token Refresh, Logout, and a GoRouter redirect when the token expires
- **Feature 2 — Real-time Power Dashboard (StreamProvider + WebSocket):** show power output (MW), frequency (Hz), and voltage in real time with `fl_chart`, auto-updating on every server push
- **Feature 3 — Daily Production Report (AsyncNotifier + Offline Cache):** fetch daily reports from the Laravel API and cache to SQLite; filter by time range and plant — usable without a network
- **Feature 4 — Machine Incident Reporting (Mutations API + Push):** submit incidents with photos, GPS, and severity, showing the full `idle/pending/success/error` cycle
- **Feature 5 — Meter Reading Entry (Cubit + MariaDB via API):** a validated form for hourly meter readings with optimistic UI updates and persistence through the Laravel API

10 Code Review, Testing & Wrap-up

- **Code Review & Best Practices:** review the code written over the 5 days and point out anti-patterns to fix
- **Testing & CI/CD:** how to unit test Cubit and Riverpod providers, plus introductory CI/CD guidance with GitHub Actions
- **Final PoC Presentation:** each team presents its complete prototype, receives feedback, and plans next steps for a real project

INTERESTED IN IN-HOUSE TRAINING? · REQUEST A QUOTE

Request a Quote

📞 02-570-8449

📞 @itgenius

📞 088-807-9770

✉️ contact@itgenius.co.th



ADD
LINE